Service Booking & Consulting Platform

Phase 1 — Complete Project Reference Guide

Course	University	Language
EECS 3311 — Software Design	York University — Lassonde	Java 17 | Maven | JUnit 5

Github Link	Ali Sheikh Rabiei	Yasamin Kheirkhahan	
https://github.com/AliSheikhRabiei/EECS3311-Service-Booking-Platform	218475095	220067328	

1. Project Overview

This system is a Service Booking & Consulting Platform that connects Clients with Consultants offering professional consulting services. It demonstrates a clean Java backend using four Gang-of-Four design patterns, an in-memory data layer, a simulated payment workflow, and a terminal-based demo UI.

What the system lets each actor do

Actor	What they can do	Use Cases
Client	Browse services, request/cancel bookings, pay, manage payment methods, view history	UC1–UC7
Consultant	Manage availability, accept/reject bookings, complete sessions	UC8–UC10
Admin	Approve consultant registrations, configure system-wide policies	UC11–UC12

Booking Lifecycle (State Machine)

Every booking travels through these states. The State pattern enforces that only legal transitions are allowed — any invalid move throws an IllegalStateException immediately.

State	Allowed next states	Who triggers it	Key rule
`REQUESTED`	CONFIRMED, REJECTED, CANCELLED	Consultant	Starting state of every booking
`CONFIRMED`	CANCELLED + payment starts	System	Consultant accepted
`PENDING_PAYMENT`	PAID, CANCELLED	PaymentService	Set before payment chain runs
`PAID`	COMPLETED	Consultant	Only state that allows complete()
`COMPLETED`	None (terminal)	—	Session delivered
`REJECTED`	None (terminal)	—	Consultant declined
`CANCELLED`	None (terminal)	—	Client cancelled / refund issued

2. How To Run

Option A — IntelliJ IDEA (Recommended)

- Open IntelliJ → click Open → select the ServiceBookingPlatform_v2 folder
- Click Trust Project when prompted
- Wait for Maven to finish importing (progress bar at bottom, ~30 seconds)
- Navigate: src → main → java → com.platform.app → Main.java
- Click the green ▶ triangle next to public static void main → Run 'Main.main()'

Option B — Maven Command Line

```
mvn compile exec:java          # run the application
mvn test                       # run all JUnit 5 tests
mvn package                    # build a runnable JAR
```

Option C — Plain javac (no Maven)

```
find src/main/java -name "*.java" > sources.txt
mkdir -p out
javac -d out @sources.txt
java -cp out com.platform.app.Main
```

NOTE	Java 17 or higher is required. If IntelliJ says 'SDK not set', go to File → Project Structure → SDK and pick Java 17+.

3. Project Folder Structure — What Each Folder Does

Folder path	Purpose
`src/main/java/com/platform/`	Root of all source code. Divided into sub-packages by responsibility.
`domain/`	Pure data objects (entities): User, Client, Consultant, Admin, Booking, Service, TimeSlot, and their enums. No business logic lives here — just fields and getters.
`state/`	State pattern classes. One file per booking lifecycle state. Controls which transitions are legal at any moment.
`service/`	Business logic layer. BookingService coordinates everything. ServiceCatalog and AvailabilityService support browsing and slot management.
`payment/`	Everything to do with payment: method types, the CoR handler chain, transaction records, receipts, and PaymentMethodService.
`policy/`	Strategy pattern interfaces and their default implementations. PolicyManager (Singleton) holds the active policies.
`repository/`	In-memory data store for bookings. Acts like a simple database using a HashMap.
`notify/`	NotificationService — prints messages to the console when events happen (booking confirmed, payment done, etc.).
`app/`	The entry point. Main.java wires all services together, runs the automated demo, then shows the interactive menu.
`src/test/java/com/platform/`	JUnit 5 test file. Tests the most critical rules: state enforcement, refund on cancellation, payment transitions.
`pom.xml`	Maven build file. Declares Java 17, JUnit 5 dependency, and the run command (mvn exec:java).

4. Every File — What It Is & What It Does

4.1 domain/ — The Core Data Objects

These files represent real-world things in the system. They hold data (fields) and have simple methods. They do NOT contain business logic — that lives in the service layer.

File	What it does
`User.java`	Abstract base class. Every person in the system (Client, Consultant, Admin) extends this. Holds id, name, and email. Cannot be instantiated directly.
`Client.java`	Represents a user who books services. Extends User. No extra fields in Phase 1 — all client behaviour is handled by the service layer (BookingService).
`Consultant.java`	Represents a service provider. Extends User. Has registrationStatus (PENDING/APPROVED/REJECTED). Has decideBooking() which calls confirm() or reject() on a booking.
`Admin.java`	System administrator. Extends User. Can approve/reject consultant registrations and swap out any of the four system-wide policies via PolicyManager.
`Service.java`	A consulting offering (e.g., 'Java Tutoring'). Has title, description, durationMin, price, and a reference to the Consultant who offers it.
`TimeSlot.java`	A specific time window (start/end LocalDateTime). Tracks whether it is available. reserve() locks it; release() frees it back for other bookings.
`Booking.java`	The central entity. Holds client, service, slot, and current BookingState. Every action (confirm, reject, cancel, paymentSuccessful, complete) is delegated to the current state object.
`RegistrationStatus.java`	Simple enum: PENDING, APPROVED, REJECTED. Used on Consultant to track admin review.
`Decision.java`	Simple enum: ACCEPT or REJECT. Passed to Consultant.decideBooking() to express the consultant's choice.

4.2 state/ — The State Pattern

These files implement the State design pattern for the booking lifecycle. Each concrete state class overrides only the transitions it allows. Everything else throws IllegalStateException automatically via AbstractBookingState.

File	What it does
`BookingState.java`	Interface defining the 5 actions: confirm, reject, cancel, paymentSuccessful, complete. Every state class implements this.
`AbstractBookingState.java`	Abstract base class. Implements ALL 5 methods with a default throw IllegalStateException. Concrete states only override what they allow — this eliminates boilerplate.

`RequestedState.java`	Initial state. Allows: confirm() → CONFIRMED, reject() → REJECTED, cancel() → CANCELLED. Everything else throws.
`ConfirmedState.java`	Consultant has accepted. Allows: cancel() → CANCELLED. (Payment start is handled by PaymentService calling setState directly.)
`PendingPaymentState.java`	Payment is in progress. Allows: paymentSuccessful() → PAID, cancel() → CANCELLED.
`PaidState.java`	Payment done. Allows ONLY: complete() → COMPLETED. This is the key lifecycle rule: cannot complete without paying first.
`RejectedState.java`	Terminal state. No overrides — all 5 methods throw via AbstractBookingState. Nothing can happen to a rejected booking.
`CancelledState.java`	Terminal state. Same as RejectedState — no further transitions possible.
`CompletedState.java`	Terminal state. Session is done. All methods throw. This booking's lifecycle is finished.

4.3 repository/ — In-Memory Data Storage

File	What it does
`BookingRepository.java`	Acts like a simple database. Stores bookings in a HashMap (bookingId → Booking). Methods: save(), findById(), findByClient(), findByConsultant(), findAll(). No SQL — everything stays in memory while the program runs.
`ConsultantRepository.java`	Acts like a simple database. SaveConsultant Stores consultants in a HashMap. FindById() Returns the Consultant with the given id Find all()Returns every stored booking

4.4 service/ — Business Logic Layer

This is where the real work happens. These classes coordinate between domain objects, repositories, payment, and policy.

File	What it does
`ServiceCatalog.java`	Holds the list of all consulting services. Methods: addService(), listAllServices(), findById(). Used for UC1 (Browse Services).
`AvailabilityService.java`	Manages each consultant's time slots. Methods: addTimeSlot(), removeTimeSlot(), listAvailableSlots(). Used during booking creation to confirm a slot is free.
`BookingService.java`	The core coordinator. Handles: createBooking() with validation, confirmBooking(), rejectBooking(), completeBooking(), cancelBooking() with policy checks and refund triggering, getBookingsForClient(). All use-case logic for UC2–UC4, UC9–UC10 runs through here.

4.5 payment/ — Payment Subsystem

Contains the Chain of Responsibility payment pipeline, payment method types, transaction records, and payment history.

File	What it does
`PaymentMethod.java`	Abstract base class for all payment types. Has owner (Client) and methodId. Declares validate() which each subclass must implement.
`CreditCardMethod.java`	Credit card. validate() checks: 16-digit number, future MM/yy expiry, 3-4 digit CVV.
`DebitCardMethod.java`	Debit card. validate() checks: 16-digit number and future MM/yy expiry.
`PayPalMethod.java`	PayPal account. validate() checks that email contains '@' and '.' in the right order.
`BankTransferMethod.java`	Bank transfer. validate() checks: account number 8-17 digits, routing number exactly 9 digits.
`PaymentHandler.java`	Interface for the Chain of Responsibility. Single method: handle(booking, method) returns a PaymentTransaction.
`AbstractPaymentHandler.java`	Base class for all handlers. Holds a 'next' reference. passToNext() forwards to the next handler in the chain.
`ValidateMethodHandler.java`	First handler in chain. Calls method.validate(). If invalid, returns a FAILED transaction immediately and stops the chain.
`ProcessPaymentHandler.java`	Second handler. Sleeps 2-3 seconds (simulated delay), generates a transaction ID, applies 10% random failure rate. If FAILED, stops here. If SUCCESS, passes forward.
`MarkPaidHandler.java`	Third (final) handler. Called only on success. Calls booking.paymentSuccessful(tx) to move booking to PAID, creates a Receipt, and sends a notification.
`PaymentService.java`	Orchestrates the payment workflow. Builds the handler chain. processPayment() first sets booking to PENDING_PAYMENT, then runs the chain. refund() creates a REFUNDED transaction using RefundPolicy. Stores all transaction history.
`PaymentMethodService.java`	Manages saved payment methods per client. In-memory Map<clientId, List<PaymentMethod>>. Methods: addMethod, removeMethod, listMethods, updateMethod.
`PaymentTransaction.java`	A record of one payment attempt. Holds transactionId, booking, amount, PaymentStatus (PENDING/SUCCESS/FAILED/REFUNDED), and timestamp.
`PaymentStatus.java`	Enum: PENDING, SUCCESS, FAILED, REFUNDED.
`Receipt.java`	Confirmation record created only when a payment succeeds. Holds receiptId, amount, method name, and timestamp.

4.6 policy/ — Strategy Pattern (Configurable Rules)

These interfaces and classes let the Admin swap out business rules at runtime without changing any other code — that is the Strategy pattern.

File	What it does
`PolicyManager.java`	Singleton. Holds the four active policy objects. Every part of the system that needs a policy calls PolicyManager.getInstance().getXxxPolicy(). The Admin changes policies here.
`CancellationPolicy.java`	Interface. Two methods: canCancel(booking, now) → boolean, cancellationFee(booking, now) → double.
`DefaultCancellationPolicy.java`	Default implementation. Allows cancellation for any booking not yet PAID/COMPLETED/REJECTED. No cancellation fee.
`RefundPolicy.java`	Interface. One method: calculateRefund(transaction, now) → double.
`DefaultRefundPolicy.java`	Default implementation. Returns 80% of the original transaction amount as a refund.
`NotificationPolicy.java`	Interface. One method: shouldNotify(eventType) → boolean.
`DefaultNotificationPolicy.java`	Default implementation. Always returns true — notify on every event.
`PricingStrategy.java`	Interface. One method: calculatePrice(service) → double.
`DefaultPricingStrategy.java`	Default implementation. Returns the base price of the service unchanged.

4.7 notify/

File	What it does
`NotificationService.java`	Sends simulated messages to users. Checks NotificationPolicy before printing. Infers the event type from the message text (e.g., if it contains 'confirmed' → BOOKING_CONFIRMED) and lets the policy decide whether to send.

4.8 app/

File	What it does
`Main.java`	Entry point and UI. Does three things: (1) setupDemoData() — creates all demo objects (admin, consultant, client, services, slots, saved credit card). (2) runHappyPathDemo() — automatically runs all 8 demo steps showing UC1-UC11 working. (3) runMenu() — drops into an interactive numbered menu so you can test any feature manually.

4.9 test/

File	What it does
`BookingStateTest.java`	JUnit 5 test class. Contains 4 focused tests: (1) bookingCannotCompleteUnlessPaid — verifies complete() throws in every state except PAID. (2) cancelPaidBookingCreatesRefundTransaction — verifies a REFUNDED transaction is created when cancelling a paid booking. (3) invalidStateTransitionsThrow — exhaustively checks that all illegal transitions throw IllegalStateException. (4) processPaymentMovesToPaidOnSuccess — uses a fast stub handler to verify the full CONFIRMED → PENDING_PAYMENT → PAID flow.

5. How the Files Connect & Work Together

5.1 The Happy Path Flow (What happens when a booking is made and paid)

#	Files involved	What happens
1	`Main → ServiceCatalog`	Client browses the catalog. ServiceCatalog.listAllServices() returns the list.
2	`Main → AvailabilityService`	Client asks for free slots. listAvailableSlots(service) filters slots where isAvailable = true.
3	`Main → BookingService → BookingRepository`	createBooking() validates consultant is APPROVED and slot is available, then calls slot.reserve(), creates a Booking in RequestedState, saves it to BookingRepository.
4	`BookingService → NotificationService`	Consultant is notified of the new request via NotificationService.notify().
5	`Main → BookingService → Booking → RequestedState`	confirmBooking() is called. Booking.confirm() delegates to RequestedState.confirm() which calls booking.setState(new ConfirmedState()).
6	`Main → PaymentService → Booking`	processPayment() is called. PaymentService immediately sets booking state to PendingPaymentState (Confirmed → PendingPayment per spec).
7	`PaymentService → ValidateMethodHandler`	Chain starts. ValidateMethodHandler calls method.validate(). For a CreditCardMethod it checks 16 digits, future expiry, valid CVV.
8	`ValidateMethodHandler → ProcessPaymentHandler`	Validation passed. ProcessPaymentHandler sleeps 2-3 seconds, generates a UUID transaction ID, randomly decides success/fail (10% fail rate).
9	`ProcessPaymentHandler → MarkPaidHandler`	Payment succeeded. MarkPaidHandler calls booking.paymentSuccessful(tx). Booking delegates to PendingPaymentState.paymentSuccessful() which sets state to PaidState.
10	`MarkPaidHandler → NotificationService`	Receipt is created. Client is notified of payment confirmation.
11	`Main → BookingService → Booking → PaidState`	completeBooking() is called. Booking.complete() delegates to PaidState.complete() which sets state to CompletedState.

5.2 Key File-to-File Connections

From	To	How they connect
`Booking`	`BookingState (interface)`	Booking holds a state field. Every action method calls state.confirm(this), state.cancel(this), etc. — the state object decides what happens.
`BookingService`	`Booking`	Creates, retrieves, and drives booking lifecycle. All use-case logic passes through BookingService before touching a Booking.
`BookingService`	`PolicyManager`	Checks CancellationPolicy before cancelling. Triggers refund via PaymentService if booking was paid.
`BookingService`	`BookingRepository`	save() and findById() after every state change so the booking is always up to date in storage.
`PaymentService`	`PaymentHandler chain`	Builds ValidateMethodHandler → ProcessPaymentHandler → MarkPaidHandler in its constructor. processPayment() calls handler.handle(booking, method).
`ProcessPaymentHandler`	`PaymentService`	Needs a reference to PaymentService to call storeTransaction() and setLastTransaction() so MarkPaidHandler can find the result.
`MarkPaidHandler`	`Booking + NotificationService`	Calls booking.paymentSuccessful(tx) to trigger PendingPaymentState → PaidState, then notifies the client.
`Admin`	`PolicyManager`	All four setCancellationPolicy/setRefundPolicy/setNotificationPolicy/setPricingStrategy calls delegate to the Singleton PolicyManager.
`NotificationService`	`PolicyManager`	Checks PolicyManager.getNotificationPolicy().shouldNotify(eventType) before printing any message.
`RequestedState`	`ConfirmedState`	When confirm() is called, RequestedState creates new ConfirmedState() and calls booking.setState(). Same pattern for all state transitions.
`ValidateMethodHandler`	`ProcessPaymentHandler`	AbstractPaymentHandler.setNext() links them. If validate passes, passToNext() forwards the call down the chain.

6. Design Patterns — Where & Why

Pattern	Files that implement it	Files that use it	Why it was used
State	BookingState, AbstractBookingState, RequestedState, ConfirmedState, PendingPaymentState, PaidState, RejectedState, CancelledState, CompletedState	Booking.java (holds state reference)	Enforces that a booking CANNOT be completed unless it is paid. Invalid transitions throw automatically — no if/else chains needed anywhere.
Chain of Responsibility	PaymentHandler, AbstractPaymentHandler, ValidateMethodHandler, ProcessPaymentHandler, MarkPaidHandler	PaymentService (builds and runs the chain)	Separates payment into independent steps: validate, process, mark paid. Each step can fail independently without affecting the others.
Strategy	CancellationPolicy, RefundPolicy, NotificationPolicy, PricingStrategy (interfaces) + Default implementations	BookingService, PaymentService, NotificationService	Lets the Admin swap rules at runtime (e.g., switch from DefaultCancellationPolicy to a StrictCancellationPolicy) without changing any service code.
Singleton	PolicyManager.java	BookingService, PaymentService, NotificationService, Admin	Ensures every part of the system reads the same policy configuration. PolicyManager.getInstance() always returns the same object.

7. Github and Team member contribution

Ali Shiekh Rabiei:
Base draft for use case diagram.
UML class diagram.
First draft and demo of the code.

Yasamin Kheirkhahan:
Use case diagram revised and improved.
Fixed bugs and errors in the code. Add different user functionality and improved the menus. Add update payment method. Added consultant repository. Add add service option.

https://github.com/AliSheikhRabiei/EECS3311-Service-Booking-Platform